

Introduction

Namespace F23.StringSimilarity

Classes

[Cosine](#)

[Damerau](#)

Implementation of Damerau-Levenshtein distance with transposition (also sometimes calls unrestricted Damerau-Levenshtein distance). It is the minimum number of operations needed to transform one string into the other, where an operation is defined as an insertion, deletion, or substitution of a single character, or a transposition of two adjacent characters. It does respect triangle inequality, and is thus a metric distance. This is not to be confused with the optimal string alignment distance, which is an extension where no substring can be edited more than once.

[Jaccard](#)

Each input string is converted into a set of n-grams, the Jaccard index is then computed as $|V1 \cap V2| / |V1 \cup V2|$. Like Q-Gram distance, the input strings are first converted into sets of n-grams (sequences of n characters, also called k-shingles), but this time the cardinality of each n-gram is not taken into account. Distance is computed as $1 - \text{cosine similarity}$. Jaccard index is a metric distance.

[JaroWinkler](#)

[Levenshtein](#)

[LongestCommonSubsequence](#)

[MetricLCS](#)

Distance metric based on Longest Common Subsequence, from the notes "An LCS-based string metric" by Daniel Bakkeland.

[NGram](#)

N-Gram Similarity as defined by Kondrak, "N-Gram Similarity and Distance", String Processing and Information Retrieval, Lecture Notes in Computer Science Volume 3772, 2005, pp 115-126.

The algorithm uses affixing with special character '\n' to increase the weight of first characters. The normalization is achieved by dividing the total similarity score the original length of the longest word.

total similarity score the original length of the longest word.

[NormalizedLevenshtein](#)

[OptimalStringAlignment](#)

[QGram](#)

[RatcliffObershelp](#)

Ratcliff/Obershelp pattern recognition

The Ratcliff/Obershelp algorithm computes the similarity of two strings as the doubled number of matching characters divided by the total number of characters in the two strings. Matching characters are those in the longest common subsequence plus, recursively, matching characters in the unmatched region on either side of the longest common subsequence. The Ratcliff/Obershelp distance is computed as $1 - \text{Ratcliff/Obershelp similarity}$.

Author: Ligi <https://github.com/dxpux> (as a patch for fuzzystring) Ported to java from .net by denmase Ported back to .NET by paulirwin to retain compatibility with upstream Java project

[ShingleBased](#)

[SorensenDice](#)

[WeightedLevenshtein](#)

Interfaces

[ICharacterInsDel](#)

As an adjunct to [ICharacterSubstitution](#), this interface allows you to specify the cost of deletion or insertion of a character.

[ICharacterSubstitution](#)